

1. Explain Risk Assessment and Management.

Risk Assessment and Management is a systematic process used in software engineering to **identify, analyze, and control potential risks** that may negatively affect a software project.



◆ 1. Risk Identification

This is the first step where all **possible risks are identified**.

- Includes **technical risks** (design issues, new technology),
- **project risks** (time, cost, resources),
- and **business risks** (market changes, financial loss).
👉 Example: Requirement changes, lack of skilled staff.

◆ 2. Risk Analysis

In this step, each identified risk is **evaluated based on**:

- **Probability** (chance of occurring)
- **Impact** (severity of damage)
👉 Helps in understanding which risks are dangerous.

◆ 3. Risk Prioritization

Risks are **ranked according to their importance**.

- High probability + high impact → **High priority**
- Low probability + low impact → **Low priority**
👉 Focus is given to the most critical risks first.

◆ 4. Risk Planning

Strategies are developed to handle risks effectively:

- **Avoid** – eliminate the risk
- **Reduce** – minimize its effect
- **Transfer** – shift risk (e.g., insurance, outsourcing)
- **Accept** – tolerate minor risks

◆ 5. Risk Mitigation

Actions are taken to **reduce or control risks**.

- Example: better testing, training, backup plans
👉 Prevents risks from becoming serious problems.

◆ 6. Risk Monitoring & Control

Risks are **continuously tracked throughout the project**.

- Identify new risks
- Monitor existing risks
- Take corrective actions when needed

✅ Conclusion

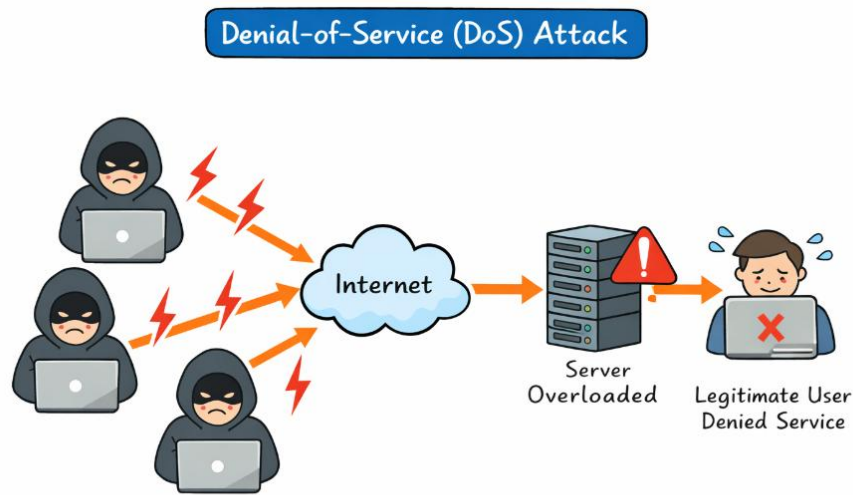
Risk Assessment and Management helps in **early detection, proper planning, and control of risks**, ensuring **successful and high-quality software development**.

2. Explain the denial- of- service with diagram.

◆ Definition

A Denial-of-Service (DoS) attack is a type of cyber attack in which an attacker makes a computer system, network, or server unavailable to legitimate users by overwhelming it with excessive requests or traffic.

◆ How It Works



1. The attacker sends a large number of requests to the target system.
2. Sometimes multiple infected systems (botnets/zombie machines) are used (called DDoS).
3. The server's resources such as CPU, memory, and bandwidth get exhausted.
4. As a result, the server becomes slow or crashes.
5. Legitimate users are unable to access the service.

◆ Characteristics

- Service Unavailability – Main goal is to deny access to users.
- High Traffic Volume – Large number of fake requests.
- Resource Exhaustion – Consumes CPU, memory, and bandwidth.
- Single or Distributed Attack –
 - DoS → single system
 - DDoS → multiple systems
- Affects Availability – Impacts the “Availability” of the CIA triad.
- Difficult to Detect – Looks similar to normal traffic in many cases.

◆ Examples

- **Flooding Attack** – Sending millions of requests to a website.
- **Ping of Death** – Sending malformed or oversized packets.
- **Bandwidth Attack** – Consuming all available network bandwidth.
- **Real-life example:** Popular websites going down due to heavy traffic attacks.

✅ Conclusion

Denial-of-Service attacks are dangerous because they disrupt services and affect users, making systems unreliable. Proper security measures like firewalls, traffic filtering, and load balancing are needed to prevent such attacks.

3. Discuss different types of risks in software development with examples.

◆ Introduction

In software development, a risk is any uncertain event that can negatively affect the project's cost, schedule, quality, or performance. Identifying different types of risks helps in better planning and control.

◆ Types of Risks

1. Technical Risks

These risks arise due to technical issues or challenges in development.

- Problems with new or untested technology
- Design complexity or integration issues

👉 Example: Using a new programming framework that developers are unfamiliar with may lead to system failure or delays.

2. Project Risks

These risks affect the project schedule, cost, and resources.

- Lack of skilled staff
- Unrealistic deadlines
- Budget constraints

👉 Example: Project delay due to shortage of developers or improper planning.

3. Business Risks

These risks impact the organization or business goals.

- Market changes
- Loss of customer interest
- Financial losses

👉 Example: Software becomes obsolete because a competitor launches a better product.

4. Requirement Risks

These risks occur due to unclear or changing requirements.

- Incomplete requirements

- Frequent changes by clients

👉 Example: Client changes system features during development, causing rework.

5. Operational Risks

These risks arise during deployment and usage of software.

- System crashes
- Maintenance issues
- User errors

👉 Example: Software fails after deployment due to improper configuration.

6. Security Risks

These involve threats to data and system security.

- Unauthorized access
- Cyber attacks (e.g., DoS attack)

👉 Example: Hackers stealing user data due to weak security.

◆ Conclusion

Different types of risks in software development can affect the success of a project. Proper risk identification and management helps in minimizing these risks and ensures successful and reliable software delivery.

4. Discuss various methods to improve software quality.

◆ Introduction

Software quality refers to how well a software product meets user requirements, reliability, performance, and security standards. Improving software quality is essential to reduce failures and increase user satisfaction.

◆ Methods to Improve Software Quality

1. Requirement Analysis

- Clearly understand and document user requirements.
 - Avoid ambiguity and incomplete specifications.
- 👉 Benefit: Reduces requirement-related errors.

2. Proper Software Design

- Use structured design techniques (modular design, architecture planning).

- Follow design principles like low coupling and high cohesion.
👉 Benefit: Makes software more maintainable and efficient.

3. Coding Standards and Best Practices

- Follow standard coding guidelines.
- Write clean, readable, and well-documented code.
👉 Benefit: Reduces bugs and improves maintainability.

4. Software Testing

- Perform different types of testing:
 - Unit Testing
 - Integration Testing
 - System Testing
 - Acceptance Testing
- 👉 Benefit: Detects and removes defects before deployment.

5. Code Reviews and Inspections

- Peer review of code to identify errors early.
👉 Benefit: Improves code quality and reduces defects.

6. Quality Assurance (QA)

- Establish processes and standards to ensure quality.
- Continuous monitoring of development activities.
👉 Benefit: Ensures consistency and reliability.

7. Use of Automated Tools

- Use tools for testing, debugging, and performance analysis.
👉 Benefit: Saves time and improves accuracy.

8. Proper Documentation

- Maintain clear documentation (requirements, design, user manuals).
👉 Benefit: Helps in maintenance and reduces misunderstandings.

9. Continuous Maintenance and Updates

- Fix bugs and improve features regularly after deployment.
👉 Benefit: Enhances long-term software quality.

◆ Conclusion

Software quality can be improved through proper planning, disciplined development practices, and continuous testing and maintenance, leading to reliable and efficient software systems.